PROJETO DE EXTENSÃO — SISTEMAS CIBERFÍSICOS

Sistema de Chamada Automatizada por RFID

Documentação de Arquitetura, Protocolo de Comunicação MQTT e Lógica de Controle (Módulos 1 e 2)

Integrantes (Hardware & Backend): Felipe Lima & Pedro Prasidos

Tópico MQTT: `pucpr/presenca/sala1`

1. Visão Geral e Escopo do Desenvolvimento

Este documento apresenta o funcionamento técnico e o fluxo de dados implementados até o momento no protótipo de chamada automatizada por RFID. O foco central da primeira versão (V1) é consolidar o ciclo completo entre o **mundo físico** (aproximação do cartão no leitor) e o **sistema digital** (transporte em rede e validação das regras de negócio em memória), sem dependência inicial de banco de dados.

O trabalho documentado abrange a integração direta entre o **Módulo 01 (Aquisição Física e Publicação)** e o **Módulo 02 (Transporte MQTT e Lógica de Controle no Backend)**, preparando o estado processado da turma para futuro consumo pelo Módulo 03 (Interface Web).

Módulo	Foco de Atuação	O que está implementado e integrado
Aluno 01 Hardware & Publisher	Captura Física (ESP32 + RC522) e Simulação de Eventos	Leitura física do identificador único (UID) via comunicação SPI no RC522, conexão Wi-Fi no ESP32 e disparo de pacotes JSON via MQTT. Inclui script simulador (`publisher.py`) para testes contínuos de carga e validação.
Aluno 02 Comunicação & Backend	Broker MQTT, Subscriber e Lógica de Controle	Assinatura do tópico no broker (`subscribe.py`), decodificação e tratamento de erros de pacotes JSON, validação de cartões cadastrados, bloqueio de leituras duplicadas e cálculo em tempo real de presentes e faltantes.
Aluno 03 Aplicação Web	Visualização e Dashboard *(Escopo Paralelo)*	Fora do escopo deste documento. Consumirá a estrutura de estado e a lista de presença geradas pelo Módulo 02.

2. Arquitetura da Solução (Hardware ao Backend)

A arquitetura separa claramente a captura do evento físico da interpretação da regra de negócio. O leitor RFID e o microcontrolador não armazenam nomes de alunos nem decidem quem está presente: sua única função é capturar o `UID` da TAG e notificar a rede. Toda a inteligência de validação reside no Backend Python.



Figura 1 — Arquitetura implementada: envio real pelo ESP32, apoio do simulador de testes e processamento centralizado via EMQX.

3. Decisões Técnicas e Evolução em Relação à Proposta Inicial

Durante o desenvolvimento prático entre os Módulos 1 e 2, três decisões importantes foram tomadas para garantir estabilidade de conexão, simplicidade no contrato de dados e agilidade nos testes:

- **Migração do Broker (Mosquitto → EMQX):** A especificação inicial previa o uso do broker Mosquitto. No início dos testes, a equipe estabeleceu conexão com o broker online do Mosquitto normalmente; contudo, após instabilidades e indisponibilidade de conexão no servidor público deles, optou-se por migrar para o broker público `broker.emqx.io` (porta `1883`), utilizando o tópico estruturado `pucpr/presenca/sala1`.
- **Otimização do Contrato de Dados (Payload JSON Enxuto):** O guia inicial sugeria o envio de três campos (`device_id`, `event: "TAG_DETECTED"` e `uid`). Na implementação atual, decidiu-se adotar um formato mais enxuto contendo apenas `device_id` e `uid`. Essa decisão técnica baseia-se no fato de que a própria publicação de um `uid` no tópico de presença já implica, de forma inequívoca, que uma TAG foi detectada pelo leitor, eliminando redundância no tráfego de rede.
- **Simulador de Disparos para Desenvolvimento Contínuo (`publisher.py`):** Enquanto a operação final ocorre com a aproximação física das TAGs no conjunto ESP32 + RC522, foi desenvolvido um script auxiliar em Python que sorteia UIDs (válidos e desconhecidos) e publica mensagens automaticamente a cada 3 segundos. Isso permitiu testar exaustivamente todas as regras do backend sem depender da passagem manual constante de cartões físicos.

> **Contrato Oficial de Mensagem MQTT (Decidido para a Versão Atual)**
>
> Tanto o firmware no ESP32 quanto o simulador `publisher.py` seguem estritamente o mesmo contrato JSON no tópico `pucpr/presenca/sala1`: enviam apenas a identificação da sala (`"device_id": "SALA_01"`) e o código hexadecimal lido do cartão (ex.: `"uid": "A3B72F91"`). A tradução desse código para o nome do estudante ocorre exclusivamente no backend.

4. Fluxograma Completo: Da Leitura RFID à Verificação no Backend

O fluxograma abaixo detalha passo a passo o caminho da informação desde o momento em que o cartão se aproxima do leitor até a árvore de decisão executada pela função `ao_receber()` no arquivo `subscribe.py`.

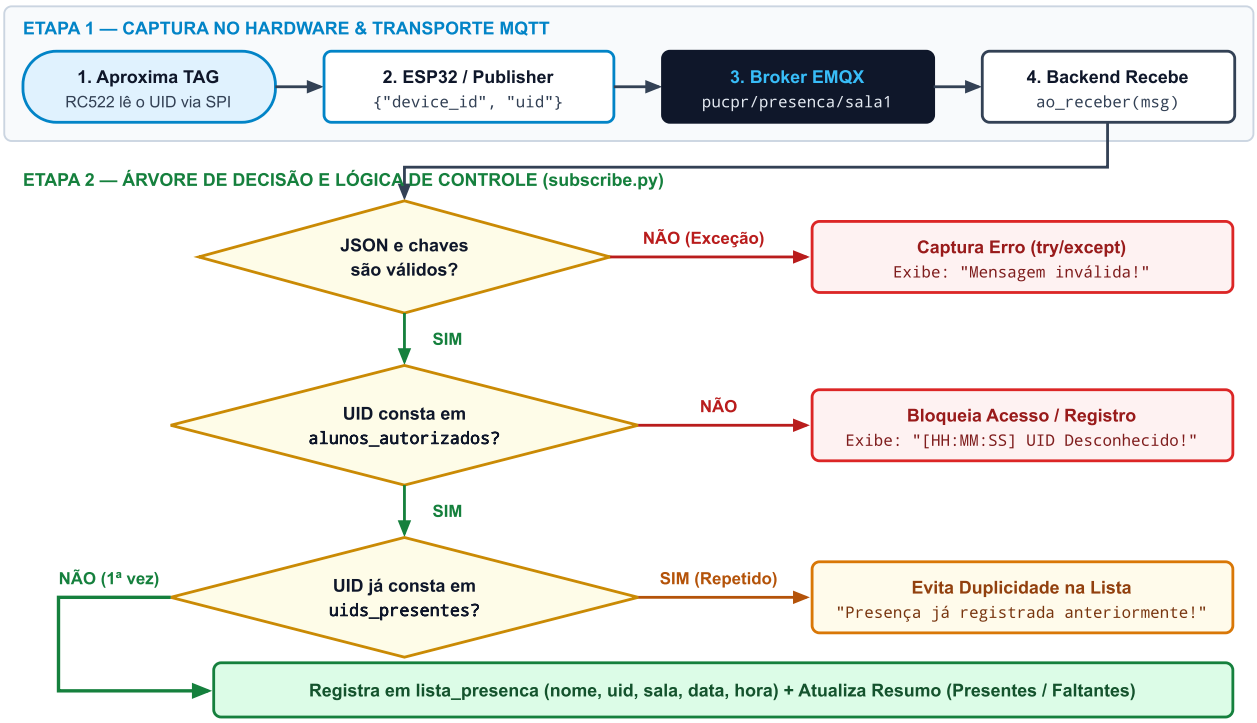


Figura 2 — Fluxograma detalhado da captura física até as três camadas de verificação do código atual.

5. Anatomia do Código Atual (Módulos 1 e 2)

Para que qualquer integrante da equipe compreenda rapidamente como o software opera sem precisar ler todo o código-fonte, a tabela abaixo resume as estruturas de dados mantidas em memória pelo `subscribe.py` e o papel de cada componente implementado.

Estrutura / Função	Tipo no Python	Finalidade na Lógica de Controle
`alunos_autorizados`	Dicionário (`dict`) 15 alunos mapeados	Base cadastral em memória que associa cada `UID` hexadecimal ao nome completo do estudante (ex.: `"A3B72F91": "Pedro Prasidos"`).
`uids_presentes`	Lista (`list`)	Guarda apenas os códigos `UID` que já registraram presença na sessão atual. Serve como trava rápida para impedir que um aluno passe o cartão duas vezes e duplique a chamada.

Estrutura / Função	Tipo no Python	Finalidade na Lógica de Controle
`lista_presenca`	Lista de Dicionários (`list[dict]`)	Estrutura principal de saída do backend. Armazena o registro completo de cada presença validada com 5 chaves: `nome` , `uid` , `dispositivo` , `data` (DD/MM/AAAA) e `horario` (HH:MM:SS).
`ao_conectar()`	Callback MQTT	Verifica se a conexão com `broker.emqx.io:1883` retornou sucesso (`reason_code == 0`) e realiza a assinatura imediata no tópico `pucpr/presenca/sala1` .
`ao_receber()`	Callback MQTT	Acionada a cada mensagem publicada no tópico. Captura o horário exato via `datetime.now()` , executa a árvore de decisão (Figura 2) e imprime o painel consolidado da turma.

6. Cenários de Validação e Saída do Backend

Abaixo estão representados o pacote real transmitido na rede (pelo ESP32 ou pelo simulador `publisher.py`) e o respectivo comportamento gerado no console do `subscribe.py` para cada um dos quatro cenários previstos pela lógica de controle:

A. Exemplo de Pacote Publicado no Tópico `pucpr/presenca/sala1` :

```
{"device_id": "SALA_01", "uid": "A3B72F91"}
```

B. Respostas do Backend (`subscribe.py`) conforme o Cenário Recebido:

```
# CENÁRIO 1: Primeira leitura de um cartão cadastrado (Sucesso)
[14:32:10] Pedro Prasidos - Presença registrada com sucesso!

========== LISTA DE PRESENÇA ==========
[{'nome': 'Pedro Prasidos', 'uid': 'A3B72F91', 'dispositivo': 'SALA_01', 'data':
'03/11/2026', 'horario': '14:32:10'}]
ALUNOS REGISTRADOS = 15 | ALUNOS PRESENTES = 1 | ALUNOS FALTANTES = 14
====================

# CENÁRIO 2: Aluno aproxima o mesmo cartão novamente (Bloqueio de Duplicidade)
[14:32:13] Pedro Prasidos - Presença já registrada anteriormente!

# CENÁRIO 3: Aproximação de TAG não cadastrada (ex.: UID "FFFFFFFF" ou "00000000")
[14:32:16] UID Desconhecido!

# CENÁRIO 4: Pacote corrompido ou fora do padrão JSON esperado
Mensagem inválida!
```

7. Ponto de Entrega para a Aplicação Web e Próximos Passos

No geral, o núcleo de funcionamento entre a ponta física/simulada e o backend está **concluído e validado**. A cadeia completa de sensoriamento, transporte via broker EMQX, filtragem de mensagens inválidas e controle de duplicidade está operando com estabilidade. Com isso, o sistema já se encontra **praticamente pronto para os primeiros testes práticos de envio e consumo de dados pelo frontend**.

Como próximo passo imediato na evolução da lógica do backend, planeja-se implementar a escrita e sincronização periódica da `lista_presenca` em **arquivos JSON locais**. Essa etapa garantirá uma persistência simples e permanente dos registros de chamada entre reinicializações do sistema — servindo como uma ponte robusta de histórico antes da introdução formal de um banco de dados relacional nas versões posteriores.